CV-Tools Filter Reference

Every filter, parameter, and caveat in one document

Filter Reference

Every filter is a plain function taking an image as its first argument and returning a new image. Registry names (the `name` column) are what appear in JSON presets and reports.

Images are RGB. `ImageLoader` converts on load and `save_image` converts back, so every filter here receives channel 0 as red — not the BGR `cv2.imread` returns. It matters for the `channel` parameter and for anything colour-aware; a luminance operation like CLAHE cannot tell the difference, which is exactly what makes the mistake easy to miss.

The measurements that return numbers rather than an image — noise, ELA, copy-move, compression, JPEG ghost and metadata — are not chain steps and are listed separately under Analysis reports.

Registry name	Function	Module	Sprint
`clahe`	`apply_clahe`	`src.filters.clahe`	1
`contrast_brightness`	`adjust_contrast_brightne`	`src.filters.contrast_brightness`	1
`auto_contrast`	`auto_contrast`	`src.filters.contrast_brightness`	1
`levels`	`adjust_levels`	`src.filters.levels`	1
`auto_levels`	`auto_levels`	`src.filters.levels`	1
`histeq`	`histogram_equalization`	`src.filters.histogram_equalization`	1
`roi_crop`	`roi_crop`	`src.filters.roi`	1
`roi_filter`	`roi_filter`	`src.filters.roi`	—
`roi_draw`	`roi_draw`	`src.filters.roi`	1
`crop`	`crop`	`src.filters.crop_resize`	1
`resize`	`resize`	`src.filters.crop_resize`	1
`rotate`	`rotate`	`src.filters.crop_resize`	1
`flip`	`flip`	`src.filters.crop_resize`	1
`sharpen`	`unsharp_mask`	`src.filters.sharpen`	2
`sharpen_laplacian`	`laplacian_sharpen`	`src.filters.sharpen`	2
`gaussian_blur`	`gaussian_blur`	`src.filters.smoothing`	2
`median_filter`	`median_filter`	`src.filters.smoothing`	2
`bilateral_filter`	`bilateral_filter`	`src.filters.smoothing`	2

`canny`	`canny_edges`	`src.filters.edge_detection`	2
`auto_canny`	`auto_canny`	`src.filters.edge_detection`	2
`sobel`	`sobel_edges`	`src.filters.edge_detection`	2
`laplacian`	`laplacian_edges`	`src.filters.edge_detection`	2
`ela`	`error_level_analysis`	`src.filters.ela`	3
`fft_spectrum`	`fft_magnitude_spectrum`	`src.filters.fft_analysis`	3
`fft_filter`	`fft_filter`	`src.filters.fft_analysis`	3
`remove_periodic`	`remove_periodic_noise`	`src.filters.fft_analysis`	3
`noise_map`	`noise_map`	`src.filters.noise_analysis`	3
`clone_detect`	`highlight_clones`	`src.filters.clone_detection`	3
`ghost`	`ghost_map`	`src.filters.jpeg_ghost`	3
`deblur_motion`	`deblur_motion`	`src.filters.motion_deblur`	3
`deblur_defocus`	`deblur_defocus`	`src.filters.motion_deblur`	3
`curves` / `s_curve`	`apply_curve` / `s_curve`	`src.filters.curves`	—
`white_balance` / `white_balance_patch` / `temperature`	`auto_white_balance` / `white_balance_from_patch` / `adjust_temperature`	`src.filters.white_balance`	—
`saturation` / `vibrance` / `desaturate` / `selective_saturation`	see module	`src.filters.saturation`	—
`color_balance` / `cmyk` / `channel_mixer`	see module	`src.filters.color_balance`	—
`invert` / `invert_channel` / `invert_luminance` / `solarize`	see module	`src.filters.invert`	—
`nl_means` / `nl_means_auto`	`nl_means_denoise`	`src.filters.nl_means_denoise`	—
`upscale`	`upscale`	`src.filters.super_resolution`	—

`local_contrast` / `detail_enhance` / `multiscale_detail` / `texture_boost`	see module	`src.filters.detail_enhancement`
`perspective` / `auto_perspective`	`correct_perspective`	`src.filters.perspective_correction`
`barrel` / `fisheye`	`correct_barrel_distortion` / `correct_fisheye`	`src.filters.fisheye_correction`
`pixel_aspect` / `fit_aspect`	`correct_pixel_aspect` / `fit_to_aspect`	`src.filters.aspect_ratio` —
`undistort`	`undistort_with_file`	`src.filters.undistort` —
`clahe_grid`	`apply_clahe_grid`	`src.filters.clahe` —
`blocking_map` / `deblock`	see module	`src.filters.compression_analysis`
`stain`	`extract_stain`	`src.filters.color_deconvolution`
`component` / `bit_plane`	`extract_component` / `extract_bit_plane`	`src.filters.component_separation`
`redact`	`redact_region`	`src.filters.redaction` —

Sprint 1 — Adjust & Correct

CLAHE — `clahe`

Adaptive contrast enhancement applied to tiles, with contrast limiting to avoid amplifying noise. The workhorse for dark or low-contrast CCTV footage.

Param	Type	Default	Notes
`clip_limit`	float	`2.0`	Higher = more local contrast, more noise
`tile_grid_size`	int or (rows, cols)	`8`	`8` means 8×8 tiles
`color_mode`	str	`'lab'`	`lab`, `hsv`, `yuv`, `channelwise`, `luminance`

`lab`, `hsv` and `yuv` equalize a single luminance channel and leave chroma alone, so colors stay stable. `channelwise` equalizes R, G and B independently and **will** shift color — use it only when you want that.

CLI: `--clahe clip=3.0 tile=8x8 mode=lab` On depth: a 10- or 12-bit source arrives as `uint16` and is equalized at 16 bits by `yuv`, `channelwise` and `luminance`. `lab` and `hsv` raise on 16-bit input, because OpenCV's conversions for those two accept 8-bit only — convert the image yourself and accept the loss, or pick a mode that holds the depth.

`apply_clahe_grid(image, clip_limits, tile_grid_sizes)` renders a labelled grid of parameter combinations for picking values quickly. Registered as `clahe_grid`, so both front ends offer it; both arguments default to a usable sweep and accept a single value. The noise cost of a given `clip_limit` varies by a factor of 1.4 to 1.9 from image to image, which is why the value to use is chosen from the board rather than guessed at a slider.

Contrast & Brightness — `contrast_brightness`

`output = (input - 128) * contrast + 128 + brightness`, then gamma.

Param	Type	Default	Notes
`brightness`	float	`0.0`	Offset, -255 to 255
`contrast`	float	`1.0`	1.0 = unchanged
`gamma`	float	`1.0`	<1 darker midtones, >1 brighter
`channel`	str or None	`None`	`r`, `g`, `b` to target one channel

CLI: `--brightness 30`, `--contrast 1.5`, `--gamma 0.8` (each is a separate chain step)

Auto Contrast — `auto_contrast`

Stretches the luminance histogram to the full range.

Param	Type	Default	Notes
`cutoff`	float	`0.0`	Percent of darkest/brightest pixels to ignore (0–50)

CLI: `--auto-contrast` or `--auto-contrast 2`

Levels — `levels`

Maps input range `[black_point, white_point]` onto `[output_black, output_white]` with a gamma curve on the midtones.

Param	Type	Default
`black_point`	float	`0`
`gamma`	float	`1.0`
`white_point`	float	`255`
`output_black`	float	`0`
`output_white`	float	`255`
`channel`	str or None	`None`

Raises `ValueError` if `black_point >= white_point`.

CLI: `--levels 20,1.0,220` (black, gamma, white)

Auto Levels — `auto_levels`

Param	Type	Default	Notes
`per_channel`	bool	`False`	`True` stretches R/G/B independently and can shift color

CLI: `--auto-levels`

Histogram Equalization — `histeq`

Global equalization — flattens the whole histogram at once. Stronger and blunter than CLAHE; prone to amplifying noise in flat regions.

Param	Type	Default	Notes
`color_mode`	str	`'lab'`	`lab`, `hsv`, `yuv`, `channelwise`, `grayscale`
`mask`	ndarray or None	`None`	Restrict the region used to build the histogram

CLI: `--histeq mode=lab`

ROI Filter — `roi_filter`

Runs another registered filter inside one region and leaves the rest of the frame as it was, because the rest is not what is being demonstrated. This is also the honest answer to a bimodal histogram: applied to the region that carries the question, a contrast operation works on the histogram that matters.

parameter	type	default	notes
`x`, `y`, `width`, `height`	int	—	The region, clipped to the image
`filter_name`	str	`'clahe'`	Registry name of the filter to run inside
`feather`	int	`8`	Width of the blend ramp in pixels; `0` for a hard edge

The edge is ramped rather than cut: a visible seam around an enhanced region is a question at the hearing. The ramp is clamped to the region, so a small ROI gets a proportionate one.

The inner filter runs with its own defaults — naming a filter *and* its parameters would need nested parameters, which the registry's flat JSON contract has no room for. Only filters that run on an image alone can go inside; the rest are refused by name rather than failing further in. A filter that resizes the region is refused too.

ROI Crop — `roi_crop`

Crops to a region, **clipped** to image bounds — an oversized region silently shrinks.

Param	Type
`x`, `y`, `width`, `height`	int

CLI: `--roi 100,100,300,200`

ROI Draw — `roi_draw`

Draws a rectangle to mark a region without altering pixels underneath (unless `filled`).

Param	Type	Default
`x`, `y`, `width`, `height`	int	—
`color`	(r, g, b)	`(255, 0, 0)`
`thickness`	int	`2`
`label`	str or None	`None`
`filled`	bool	`False`
`alpha`	float	`0.3`

CLI: `--draw-roi 265,295,120,46`

Crop — `crop`

Same geometry as `roi_crop`, but **raises** `ValueError` when the region falls entirely outside the image instead of returning a clipped result. Use it when an out-of-bounds crop should be an error.

CLI: `--crop 100,100,300,200`

Resize — `resize`

Param	Type	Notes
`width`	int or None	Alone, height follows aspect ratio
`height`	int or None	Alone, width follows aspect ratio
`scale`	float or None	Used when width/height are absent
`interpolation`	str	`auto`, `nearest`, `bilinear`, `bicubic`, `lanczos`, `area`

`auto` picks `INTER_AREA` for downscaling and `INTER_LANCZOS4` for upscaling. For forensic work prefer `nearest` when you need to inspect pixels without resampling.

CLI: `--resize 800x600`, `--resize 800x`, `--resize x600`, `--resize 50%`, `--resize 0.5`, combined with `--interpolation lanczos`.

Rotate — `rotate`

Expands the canvas so no content is cropped.

Param	Type	Default
`angle`	float	— (degrees, counter-clockwise)
`center`	(x, y) or None	`None` = image center
`scale`	float	`1.0`
`border_mode`	str	`'constant'` (`replicate`, `reflect`, `wrap`)
`border_value`	(r, g, b)	`(0, 0, 0)`

CLI: `--rotate 90`

Flip — `flip`

Param	Type	Values
`direction`	str	`horizontal`, `vertical`, `both`

CLI: `--flip horizontal`

Sprint 2 — Enhance

Unsharp Mask — `sharpen`

Subtracts a blurred copy to isolate detail, then adds it back scaled by `amount`.

Param	Type	Default	Notes
`amount`	float	`1.0`	0 = no change; above 2 usually looks artificial
`radius`	float	`1.0`	Blur sigma. Small = fine detail, large = local contrast.
`threshold`	int	`0`	Minimum local contrast (0–255) before a pixel is sharpened

`threshold` is the noise control: raise it and smooth areas — where noise lives — are left alone while genuine edges still get sharpened. Sharpen **after** denoising, never before.

CLI: `--sharpen amount=1.5 radius=1.0 threshold=4`

`sharpen_grid(image, amounts, radii)` renders a labelled parameter grid, like `apply_clahe_grid`.

Laplacian Sharpen — `sharpen_laplacian`

`output = input - strength × laplacian(input)`. Harsher and more noise-sensitive than an unsharp mask, but needs no radius choice.

Param	Type	Default
`strength`	float	`1.0`
`kernel_size`	int	`3` (odd, 1–31)

CLI: `--sharpen-laplacian strength=1.0 kernel=3`

Gaussian Blur — `gaussian_blur`

Param	Type	Default	Notes
`radius`	float	`2.0`	Gaussian sigma in pixels

`kernel_size`	int	`0`	0 derives the kernel from the radius — normally what you want

General-purpose smoothing. Blurs edges along with noise; prefer bilateral when edges matter.

CLI: `--gaussian 1.5` (bare `--gaussian` uses 2.0)

Median Filter — `median_filter`

Param	Type	Default	Notes
`kernel_size`	int	`3`	Odd, 3 or greater

The standard remedy for salt-and-pepper / impulse noise. Output only ever contains values already present nearby, so plateaus and edges survive intact where a blur would smear them.

CLI: `--median 3` (bare `--median` uses 3)

Bilateral Filter — `bilateral_filter`

Param	Type	Default	Notes
`diameter`	int	`9`	Neighbourhood diameter; 0 derives it from `sigma_space`
`sigma_color`	float	`75.0`	Color-difference tolerance
`sigma_space`	float	`75.0`	Spatial extent

Weights neighbours by both distance and color similarity, so it smooths sensor noise inside regions without bleeding across boundaries. The slowest of the three smoothing filters.

CLI: `--bilateral d=9 color=75 space=75`

Sprint 2 — Analyze

All four edge detectors return a **single-channel** uint8 map, so they convert a color image to grayscale mid-chain.

Canny — `canny`

Param	Type	Default	Notes
`low_threshold`	float	`100`	Weak edges survive only if connected to a strong one
`high_threshold`	float	`200`	A 1:2 or 1:3 low:high ratio is the usual starting point
`aperture_size`	int	`3`	3, 5, or 7
`l2_gradient`	bool	`False`	Exact L2 magnitude instead of the cheaper L1
`blur_sigma`	float	`0.0`	Gaussian pre-blur — noisy footage needs one

Output is binary (0 or 255).

CLI: `--canny 50,150`, with `--blur-first 1.5` to set the pre-blur.

Auto Canny — `auto_canny`

Derives both thresholds from the image's median intensity, which suits batches of frames whose exposure varies. Falls back to 50/150 when the median collapses both thresholds onto the same value (a near-black or near-white frame).

Param	Type	Default	Notes
`sigma`	float	`0.33`	Spread around the median, 0–1. Larger keeps more edges.
`blur_sigma`	float	`0.0`	Gaussian pre-blur

CLI: `--auto-canny` or `--auto-canny 0.4`

Sobel — `sobel`

Param	Type	Default	Notes

`dx`	int	`1`	Horizontal derivative order (0 or 1)
`dy`	int	`1`	Vertical derivative order (0 or 1)
`kernel_size`	int	`3`	Odd, 1–7
`normalize`	bool	`True`	Stretch the result to fill 0–255

With both `dx` and `dy` set you get the gradient magnitude; with one you get that single directional derivative. Continuous-valued, unlike Canny's binary output.

CLI: `--sobel dx=1 dy=1 kernel=3`

Laplacian — `laplacian`

Param	Type	Default
`kernel_size`	int	`3` (odd, 1–31)
`normalize`	bool	`True`
`blur_sigma`	float	`0.0`

Responds to intensity change in all directions at once — and to noise just as eagerly, so a small `blur_sigma` is usually worth it.

CLI: `--laplacian kernel=3 blur=1.0`

Histogram (not a chain step)

`histogram.py` analyzes rather than transforms, so it is exposed as CLI output options instead of registry filters.

Function	Returns
`compute_histogram(image, bins=256, normalize=False)`	Dict of channel name → bin counts
`histogram_stats(image)`	Per-channel mean, median, std, min, max, p1, p99, clipping %
`dynamic_range_used(image)`	Fraction of 0–255 spanned between p1 and p99
`render_histogram(image, ...)`	RGB chart image

Clipping percentages are the forensically important part: pixels stuck at 0 or 255 have lost their original values, and no enhancement recovers them. A low `dynamic_range_used` means levels or CLAHE still has room to work.

`render_histogram` accepts `width`, `height`, `bins`, `log_scale` (reveals sparse tails a linear plot flattens to nothing), `show_grid`, `background`, and `channels` to restrict which curves are drawn.

CLI: `--histogram chart.png`, `--histogram-log`, `--hist-stats`

`edge_density(edges, threshold=0)` gives the fraction of pixels carrying an edge. It compares focus across frames of the same scene, but it is not a general sharpness measure — blurring an image whose only feature is one strong edge spreads that edge over more pixels and raises the density.

Sprint 3 — Forensic

Read this before using any of them. These filters find things *worth examining*. None of them establishes that an image was manipulated, and each has a failure mode that produces convincing-looking evidence for a false conclusion. The caveats below are part of the tool, not disclaimers around it.

Error Level Analysis — `ela`

Re-compresses the image as JPEG and amplifies the difference. A region with a different compression history than its surroundings may show a different error level.

Param	Type	Default	Notes
`quality`	int	`90`	Re-compression quality, 1–100. Aim near the original's.
`scale`	float	`0`	Brightness multiplier; 0 auto-scales so the peak error hits 255
`grayscale`	bool	`False`	Collapse per-channel error into one channel

Limits. Only meaningful on a JPEG original — re-saving as PNG, or a second full-image JPEG save, erases the signal completely. Bright areas track edge density and texture as much as editing history, so busy regions always look hot. A clean map does not mean the image is authentic.

CLI: `--ela quality=90 gray=true`, `--ela-stats [QUALITY]`

`ela_stats(image, quality, block_size)` returns the mean/max error, a per-block mean grid, and the hottest block with a z-score saying how far it stands above the rest. `recompress(image, quality)` exposes the JPEG round-trip on its own.

FFT Magnitude Spectrum — `fft_spectrum`

Param	Type	Default	Notes
`log_scale`	bool	`True`	Without it the DC term dwarfs everything and the plot is one dot
`normalize`	bool	`True`	Stretch to fill 0–255

DC is at the centre. Periodic structure — interlacing, halftone screens, scanner banding — appears as discrete bright points away from that centre.

CLI: `--fft log=true`

Frequency Filter — `fft_filter`

Param	Type	Default	Notes
`filter_type`	str	`'lowpass'`	`lowpass`, `highpass`, `bandpass`
`cutoff`	float	`30.0`	Radius in pixels from DC
`cutoff_high`	float	`0.0`	Upper radius, bandpass only
`soft`	bool	`True`	Gaussian-edged mask. A hard edge causes ringing that looks like real image content.

Returns single-channel output.

CLI: `--fft-filter type=highpass cutoff=20`

Periodic Noise Removal — `remove_periodic`

Finds isolated spectral peaks and notches them out, removing a repeating pattern with far less damage than a blur would do. The mirrored peak is notched too, since a real image's spectrum is symmetric about DC.

Param	Type	Default	Notes
`peaks`	list or None	`None`	Detected automatically when omitted
`notch_radius`	float	`4.0`	Radius of the Gaussian notch on each peak
`min_radius`	float	`10.0`	Ignore this radius around DC — low frequencies are the image itself
`threshold`	float	`4.0`	Standard deviations above the local background for a peak to count

Some residual pattern usually survives at the image borders: the FFT treats the image as tiling the plane, and the discontinuity at the edges spreads energy across the spectrum.

CLI: `--remove-periodic notch=4`

`detect_periodic_peaks(image, ...)` returns the peaks alone, so you can inspect them before notching.

Noise Map — `noise_map`

Param	Type	Default	Notes
`block_size`	int	`32`	Smaller localises better but estimates each block less reliably
`normalize`	bool	`True`	Stretch to fill 0–255
`upscale`	bool	`True`	Resize the block grid back to the input's dimensions

Bright means noisier. Sensor noise should be fairly even across an untouched frame, so a region that differs markedly came from somewhere else — a different camera, a different resize, or a denoise pass applied to that region alone. Heavy texture also raises the reading.

CLI: `--noise-map 32`, `--noise-stats`

`estimate_noise(image)` returns the global sigma via Immerkaer's method; `estimate_snr` gives dB (using the image's own std as signal, so a flat image scores low however clean); `noise_report` adds per-block statistics and a `uniformity` ratio.

Clone Detection — `clone_detect`

Finds regions duplicated from elsewhere in the same image. Blocks are described by their low-frequency DCT coefficients, sorted so near-identical blocks become neighbours, and a shift shared by many pairs is reported as a duplicated region.

Param	Type	Default	Notes
`step`	int	`1`	**Only shifts that are multiples of this are detectable**
`block_size`	int	`16`	Side length of each compared block
`coefficients`	int	`4`	Size of the top-left DCT square kept as the descriptor
`quantization`	float	`4.0`	Descriptor rounding; larger tolerates more compression, matches more falsely
`min_distance`	float	`0`	Minimum separation for a pair to count; 0 uses 2×`block_size`
`min_matches`	int	`8`	Pairs needed before a shift is reported

`min_variance`	float	`12.0`	Featureless blocks below this are skipped
`max_blocks`	int	`300000`	Memory guard; raises rather than allocating gigabytes

The `step` constraint is the one that catches people out. Blocks are sampled on a grid of that stride, so a region moved by 190 pixels is invisible to a stride of 8 — the copy is sampled at a different phase from the original and the descriptors do not match. The default of 1 is exhaustive. Raising it is a quick screening pass, not a search.

On a large image, crop to a region with `--roi` rather than raising `step`.

Limits. Genuine repetition — brick walls, windows, tiled floors, text — is duplication, and will be reported. This locates duplication, not intent.

CLI: `--clone-detect block=16 step=1 matches=8 variance=12`, `--clone-stats`

`detect_copy_move` returns the full result dict (mask, shift vectors, block counts); `draw_clone_regions` tints it onto the image.

JPEG Ghost Detection — `ghost`

Recompresses the image across a range of JPEG qualities and diffs each pass against the source, the same trick ELA uses once. Re-quantising an already-JPEG'd region at its own prior quality is nearly lossless, so a region's error dips at the quality it was last saved at — the "ghost".

You have to say which region. This does not search for one, and the reason is measured rather than assumed. Across 18 untouched CCTV frames, the most separated region an automatic search can find scores –0.53 on average, while the same frames carrying a genuine Q55 paste score –0.44 — untouched frames separate *more strongly* than forged ones, because a flat wall forms a large coherent low-difference region at some quality in every frame. A search returns texture, not history. Given the region, the same measurement is decisive: –0.34 at the true quality against –0.02 for an untouched control.

Mark the region — the desktop viewer fills `x, y, width, height` from a drag — and this reports what it was compressed at.

Param	Type	Default	Notes
`qualities`	list[int]	`50,55,...,100`	Ascending quality steps to sweep
`block_size`	int	`16`	Side length of the analysis blocks
`region`	(x,y,w,h)	`None`	The region to interrogate. Without it nothing is claimed
`upscale`	bool	`True`	`ghost_map` only: resize the block grid back to the input's dimensions

Measured accuracy. On 12 real CCTV frames carrying a known Q55 paste: found in 10, and 2 untouched frames called positive. Read a hit as a pointer to inspect, never as a finding. The recovered quality lands within one sweep step of the truth, so treat it as a neighbourhood rather than a number.

Limits. A uniform JPEG resave of the whole composite is a blind spot: every block then shares one true last quality, and its trivially-near-zero dip there swamps any subtler trace of what a region was compressed at before a splice. The technique reads a composite that was never unified by a later full-frame JPEG save — a PNG built from JPEG sources is the common case it catches. Flat, low-texture regions dip only shallowly at every quality and read as ambiguous by design. The 0.10 threshold was calibrated on one camera; re-derive it before leaning on it elsewhere.

CLI: `--ghost block=16 min=50 max=100 step=5`, `--ghost-stats`

`ghost_sweep` returns the normalised difference at every quality — the raw material, and the form worth looking at. `ghost_map` returns the single sweep frame carrying the most structure, dark where the pixels match that quality. `ghost_report` names the region's quality, the separation that decided it, and every quality's score so the verdict can be checked rather than taken.

> A previous version of this filter took each block's *global* minimum across the sweep. That cannot work: the difference curve falls monotonically towards quality 100 for almost every block, so the minimum lands at the top of the sweep whatever the block's history, and a clean frame reported 42% of its blocks as outliers. Presets and reports written before that fix record `dominant_quality` and `outlier_count`, which no longer exist.
>
> **On filters that find nothing.** `clone_detect` and `auto_perspective` return the image unchanged when they detect no duplicated region and no quadrilateral. That is the right contract for a chain step — an image goes in, an image comes out, and a preset replays identically — but it means "found nothing" and "did nothing" look the same in the viewer. Use the matching report to tell them apart: `--clone-stats` says *no duplicated regions found* explicitly, and the GUI's Analysis tab shows the same line.

Metadata Forensics (not a chain step)

Reads the container rather than the pixels: EXIF tags, JPEG application segments, and whether what the metadata claims matches what the image actually is. `metadata_report(path)` takes a file path, not an image, so it is a stats flag rather than a filter.

Check	Severity	What it means
`editing_software`	flag	`Software` names a known editor (matched against `EDITOR_SIGNATURES`, so camera firmware strings do not trigger it)
`modified_after_capture`	flag	`DateTime` is later than `DateTimeOriginal` — written again after the shutter fired
`timestamp_disorder`	flag	`DateTimeDigitized` precedes `DateTimeOriginal`, which capture order forbids
`dimension_mismatch`	flag	EXIF's recorded dimensions disagree with the actual ones — resized or cropped since capture

`photoshop_segment`	flag	An APP13 Photoshop resource block is embedded
`thumbnail_mismatch`	flag	The embedded EXIF thumbnail's content disagrees with the main image
`no_exif`	info	A format that normally carries EXIF has none
`no_camera_identification`	info	EXIF present but no `Make` or `Model`
`xmp_segment`	info	An XMP packet is embedded, which often records an editing history the EXIF does not

This is the cheapest check available and the easiest to defeat. Metadata is plain text in a header: anyone can edit or strip it, and most messaging and social platforms strip it wholesale on upload. So a clean header proves nothing — it is the normal state of a file that has been through WhatsApp — and an editor's name proves nothing either, since cropping, rotating and format conversion all leave one.

The contradictions are the part worth attention. A tag that disagrees with the pixels, or with another tag, is harder to produce by accident than a suspicious-looking name is.

The embedded thumbnail is one contradiction editors routinely leave behind. JPEGs carry a second, small copy of the image in EXIF's IFD1 for previews, and an editor that replaces the pixels has no reason to regenerate it — cropping, splicing, or swapping the subject can leave the thumbnail still showing the original scene. `check_thumbnail_mismatch` extracts it and compares its content against the main image with a cheap perceptual hash (an 8x8 average-hash), tolerant of the thumbnail's own recompression but not of a genuinely different picture. Absence of a thumbnail is not itself a finding — plenty of ordinary files never had one.

CLI: `--metadata-stats`

`read_exif` returns the tags as a plain dict; `detect_editing_software` and `check_timestamps` are the individual checks, usable on an EXIF dict you already hold. `extract_thumbnail` returns the embedded thumbnail's raw JPEG bytes, or `None`.

Wiener Deblurring — `deblur_motion`, `deblur_defocus`

Inverts a known blur. Naive inversion divides by the blur's frequency response, which is near zero at some frequencies, so noise there is amplified without limit. The Wiener filter tempers this: `F = G · conj(H) / (|H|² + K)`, where `K` is `noise_power`.

Param	Type	Default	Notes
`length`	float	`15.0`	Motion extent in pixels (`deblur_motion`)
`angle`	float	`0.0`	Motion direction in degrees, 0 = horizontal (`deblur_motion`)

`radius`	float	`5.0`	Defocus circle radius (`deblur_defocus`)
`noise_power`	float	`0.01`	Raise on noisy footage to trade sharpness for stability

The input is reflect-padded before the transform and cropped afterwards, which keeps the FFT's wraparound — where the left edge convolves with the right — out of the result.

Limits. You must supply the correct PSF; a guessed length or angle produces confident-looking detail that was never recorded. Deconvolution also assumes one uniform blur across the frame, so a scene where only one object moved needs that object isolated first with `--roi`.

CLI: `--deblur length=15 angle=30 noise=0.01`, `--deblur-defocus radius=5 noise=0.01`

`motion_blur_psf` and `defocus_psf` build the PSFs; `apply_psf` applies one forward, which is useful for previewing what a PSF means. `wiener_deconvolution` takes an arbitrary PSF.

Because the true PSF cannot be read off an image reliably, `deblur_sweep(image, lengths, angles)` renders a labelled grid across the parameter space to judge by eye, and `focus_score(image)` ranks results by the variance of the Laplacian.

Sprint 3 — Multi-frame (not chain steps)

Frames come from a video (`--frames N` on a video file, starting at `--frame`) **or from a directory of stills** (`--frames N` on a folder), which is how exported CCTV evidence usually arrives.

Super-resolution is worth about half a decibel, and only from frames that are already almost aligned. Measured against a known original, reconstructing 2x from frames carrying ideal sub-pixel offsets:

frames	gain over a bicubic upscale
2	+0.23 dB
4	**+0.64 dB**
8	+0.56 dB
16	+0.55 dB

The gain reaches its useful level at about four frames and holds there — more frames do not add much, but they no longer take anything away. They used to: before the registration was refined on an upsampled correlation surface, sixteen frames scored –**0.10 dB**, worse than not reconstructing at all, because each frame's registration error was smeared into the result. Real CCTV snapshots seconds apart shift by tens or hundreds of pixels and `super_resolve` refuses them outright — it is a tool for a burst, not for a sequence of events.

Averaging only reduces noise where the scene holds still. The improvement is `sqrt(N)` for noise that is independent between frames, and measurably less for anything else:

source	16 frames	ratio
synthetic independent noise	7.93 → 2.03	**3.90×** (√16 = 4.00)
a static scene on video	10.94 → 3.39	3.22×
motion-event snapshots seconds apart	1.73 → 1.59	1.08×

The first is the law working exactly. The second falls short because scene texture and the codec's own quantisation are correlated between frames and do not average away. The third barely moves because the scene itself changed — that is not a case for `mean` at all, and `median` is the method for it, which removes what moved rather than smearing it.

`frame_averaging.py` consumes a *sequence* of frames and produces one image, so it runs before the filter chain rather than inside it. On the CLI this is `--frames N`, with `--frame` selecting the start index and `--frame-step` the stride.

Function	CLI method	What it does
`average_frames(frames, weights=None)`	`mean`	Suppresses random noise; noise falls with the square root of the frame count

`median_frames(frames)`	`median`	Removes anything present in fewer than half the frames, reconstructing the background
`integrate_frames(frames, gain, auto_scale)`	`integrate`	Accumulates light from very dark footage without amplifying noise the way gain would
`sharpest_frames(frames, count)`	`sharpest`	Ranks frames by focus; the CLI averages the best-focused half

All of them assume the frames are **aligned**, and none of them can tell that they are not — a moving camera just returns something softer, with nothing to say why.

`frame_difference(a, b, amplify)` gives the absolute difference between two frames, for isolating what moved.

CLI: `--frames 24 --frame-method median --frame-step 5`

Stabilisation — `--stabilise`

`stabilise.py` brings a sequence into a common alignment, which is what makes the methods above usable on footage that was not shot on a tripod. Measured on a shaky, noisy 16-frame sequence against the clean original:

	PSNR
one noisy frame	17.86 dB
averaged not aligned	22.01 dB
averaged after aligning	**28.27 dB**

Averaging alone recovers 4 dB of the noise and then stalls, because it is now fighting the camera's own motion. Aligning first recovers another 6 dB — the difference between a smeared plate and a legible one.

Motion models. Pick the least freedom the camera actually had; a model with more will fit noise happily.

Model	For
`translation`	A locked-off camera on a shaky mount
`euclidean`	Handheld — shake plus roll (the default)
`affine`	Handheld with some scale change
`homography`	A pan or zoom across a broadly flat scene

Methods. `features` matches ORB keypoints and survives large motion but needs texture; `ecc` maximises correlation, is sub-pixel accurate, and is a *local* search that only converges for small motion unless it is given a starting point. `auto` seeds `ecc` from the feature estimate and so gets both, falling back to whichever half succeeded.

This is not the same job as `super_resolution.estimate_shifts`, and the two are not interchangeable: `estimate_shifts` measures pure translation to a small fraction of a pixel, which is what a reconstruction needs from a nearly-static camera. `align_frames` handles rotation, scale and perspective at whole- or near-pixel accuracy, which is what makes a moving camera's frames combinable at all. Stabilise first, then reconstruct.

> **A bad alignment smears, and hides that it did.** A frame that could not be matched is worse than a frame left out, because averaging conceals it — the result simply looks softer. So every frame comes back with a confidence, frames below `--stabilise-min-confidence` are dropped rather than warped on a guess, and `--report` records which. Note that confidence is not one scale: from `ecc` it is the correlation coefficient, from `features` the RANSAC inlier ratio. Both run 0 to 1 and more is better, but a 0.7 from one is not a 0.7 from the other.

The output is **cropped to the region every frame covers**. Aligning slides and turns each frame, so its far edge leaves a strip with no data behind it; averaging across that strip mixes real pixels with black and darkens the border, which looks like vignetting and is not. The border is data the sequence does not have.

CLI: `--stabilise [MODEL] --stabilise-method auto|ecc|features --stabilise-reference N --stabilise-min-confidence C`. `--stabilize` spellings work too.

Remaining catalogue

Each module's own docstring carries the full reasoning; this is the summary and the caveats that matter most.

Adjust

`curves` — control-point tonal curve, interpolated with a monotonic (PCHIP) spline so the mapping can never double back and invert the tonal order the way a plain cubic can. Presets: `linear`, `brighten`, `darken`, `contrast`, `reduce_contrast`, `lift_shadows`, `film`. CLI: `--curves preset=lift_shadows` or `--curves points=0:0,128:170,255:255`.

`white_balance` — each automatic method assumes something about the scene, and fails when it does not hold: `gray_world` (the average is neutral — fails when one colour dominates), `white_patch` (the brightest pixels are white — fails on a blown highlight), `shades_of_gray` (a compromise, and the default). When the scene contains something known to be neutral, `--wb-patch X,Y,W,H` measures it instead of guessing.

`saturation` / `vibrance` — vibrance weights the boost towards muted colours so vivid ones do not clip into flat blocks. Note the falloff is *proportional*: a mid-saturation colour can still gain more raw saturation than a nearly-neutral one.

`color_balance` — shifts shadows, midtones and highlights independently, with overlapping Gaussian weights so ranges blend rather than band. Useful when two light sources cast differently by brightness. `preserve_luminosity` keeps overall brightness fixed so only colour moves.

`invert` — `invert_luminance` flips brightness while keeping hue, which sometimes makes faint dark-on-dark detail readable where raising brightness only washes it out.

Enhance

`nl_means` — averages patches that *look alike* wherever they sit, so repeating texture reinforces rather than smooths away. The slowest denoiser here; cost grows with the square of `search_window`. Set `h` from measured noise via `estimate_h`, or use `--nl-means-auto`. `nl_means_denoise_frames` uses neighbouring frames as extra evidence without the smearing plain frame averaging causes on moving objects.

`super_resolution` — the distinction here matters more than anywhere else in the toolkit. `upscale` **interpolates and adds no information**; a plate unreadable at native resolution stays unreadable enlarged. `super_resolve` genuinely recovers detail, because sub-pixel motion between frames samples the scene on different grids. It needs real sub-pixel motion — `super_resolve_report` tells you whether a sequence has any.

Phase correlation drives the alignment, and it needs broadband detail. A strongly periodic scene (tiles, brickwork, a fence) produces several correlation peaks of similar height and the measured offset can be meaningless rather than merely imprecise.

How much it is worth. Measured against ground truth, on frames shifted by known amounts and downsampled, comparing reconstruction at 2x against bicubic interpolation of one frame:

Frame motion	Gain over interpolation
offsets covering the half-pixel grid	**+1.4 dB**
whole-pixel steps only	+0.6 dB

small random jitter (±0.3 px)	+1.0 dB
large random jitter (±3 px)	–0.1 dB

The gain comes from the frames sampling *between* each other's pixels, and having motion is not the same as having the right motion. The last row is the honest caution: a sequence can pass `super_resolve_report`'s `usable` check — which asks only whether ≥2 frames carry a fractional offset — and still reconstruct no better than an upscale. Treat `usable` as necessary rather than sufficient, and compare against `--upscale` before relying on the result.

> **`--stabilise` and `superres` work against each other.** Aligning a stack to sub-pixel accuracy removes precisely the motion reconstruction feeds on. The combination is measured rather than forbidden: a stabilised stack reports little sub-pixel motion and the CLI warns. Stabilise when *combining* frames; do not stabilise when *reconstructing* from them. A sequence with both large camera motion and recoverable sub-pixel detail would need a full motion model inside the reconstruction, which `estimate_shifts` (translation only) does not provide.

CLI: `--frames 12 --frame-method superres --sr-scale 2 --sr-sharpen 0.6 --sr-max-shift 8`

`detail_enhancement` — `local_contrast` is a large-radius unsharp mask, which is what most "clarity" sliders are. `enhance_detail` is edge-preserving, so it lifts texture without halos. `multiscale_detail` boosts frequency bands independently.

Correct

`perspective` — four-point rectification. Pass the surface's known real-world ratio (`KNOWN_RATIOS`: `a4_portrait`, `a4_landscape`, `us_letter`, `credit_card`, `plate_eu`, `plate_us`, `square`) because estimating it from a perspective view is unreliable. `find_document_corners` detects a rectangular surface automatically; it returns None on a cluttered scene, which is the expected outcome rather than an error.

`fisheye_correction` — `barrel` uses the polynomial radial model (hand-tuned, fine for moderate wide-angle); `fisheye` uses the equidistant model for true dome cameras. Both *estimate* the distortion. When the camera is available, calibrate instead.

`aspect_ratio` — SD video does not use square pixels; displayed uncorrected, everything is stretched and any measurement is wrong in one axis. `PIXEL_ASPECT_RATIOS` covers PAL, NTSC, HDV and anamorphic. `fit_to_aspect` mode `pad` is the only one that alters neither geometry nor content.

`undistort` — the defensible route: derive the camera's actual intrinsics from chessboard photographs, then invert exactly that. `CameraCalibration.is_reliable` checks the reprojection error is under one pixel. A calibration is specific to one camera at one zoom and focus; applying another camera's is worse than applying none, because the result looks plausible while being geometrically wrong.

Analyze

`compression_analysis` — `blockiness_score` compares intensity steps across the 8-pixel JPEG grid against steps elsewhere. `estimate_jpeg_quality` reads the quantisation tables directly from a JPEG, which is exact rather than inferred — but only while the file is still a JPEG.

The measure assumes photographic content. An image dominated by hard synthetic edges that miss the block grid inflates the interior term and can read as no blocking whatever its history. Strong blocking means heavy compression, nothing more; re-saving normalises the grid and erases any local difference.

Special

`color_deconvolution` — separates overlapping colorants (ink over print, stamp over signature) by solving in optical density, where absorption is additive. At most three colorants, since three channels give three equations, and colorants with near-parallel colour vectors separate poorly. `estimate_stain_vector` measures a vector from a patch of one colorant alone, which is the reliable way to build a separation for an unknown ink.

`component_separation` — detail invisible in a colour composite is often plain in one component. Colour spaces (`rgb`, `hsv`, `hls`, `lab`, `luv`, `ycrcb`, `yuv`, `xyz`), frequency split (base versus detail), and bit planes. Structure in the low bit planes is notable: natural sensor noise has none, so a pattern there suggests hidden data or a pasted region.

`nl_means` strength. `nl_means_auto` sets `h` to 0.6 of the measured noise sigma, which was chosen by measurement: denoising five images at two noise levels each and scoring against the clean original gave a mean gain of +2.23 dB at 0.6, against **–3.93 dB at 3.0** — the value the filter used to use. At three times sigma it removed more picture than noise and scored below the untouched input on eight of ten cases.

How much it helps depends on the scene. A smooth wall gains 6 dB; a densely textured subject — fur, foliage, gravel — loses at every strength, because its detail sits at the same scale as the noise. On a static sequence, averaging frames beats denoising one of them by a wide margin (36.9 dB against 25.0 dB on a seven-frame test).

`redaction` — the one operation that must not be undoable, and the obvious methods fail. **Blurring is reversible** — it is a known convolution, and this toolkit's own Wiener deconvolution will undo it. **Pixelation is reversible for short known-alphabet text** — rendering every candidate plate and matching block means is a documented, cheap attack. Only `fill` and `noise` discard the original pixels; `fill` is the default and the only method for a document intended for release. `verify_redaction` correlates each region against the original and reports whether the content actually went.

`noise` draws fresh noise on every run unless `seed` is given, so the same preset produces a different image each time. The redaction is equally effective either way, but a result that cannot be reproduced is not evidence — set `seed` when the chain has to replay exactly. Seeding weakens nothing: the original pixels are discarded regardless, so knowing the noise recovers none of them.

`annotate` — arrows, shapes, text, and calibrated measurement. `Scale` converts pixels to units; `measure_distance` (1D), `measure_area` (2D, shoelace formula), `draw_measurement`, `draw_area_measurement` and `draw_scale_bar` present them.

A scale is valid only for the plane it was measured in. A ruler on the ground calibrates distances on the ground and says nothing about a sign three metres behind it, which is further from the camera and smaller per pixel. Correct perspective first. Annotations are drawn on a copy — keep the unannotated original, since a marked-up image is a figure, not evidence.

Measurement — `measure`, `measure_area`, `scale_bar`

The chain steps that wrap `annotate`. A `Scale` cannot travel through a JSON preset, so these take **the two ends of something whose length you know** — a number plate, a door, a ruler left in the scene — rather than a pixels-to-units number you would otherwise have to compute by hand, which is the step most likely to go wrong.

Parameter	Meaning
`point_a`, `point_b`	What is being measured (`measure`)
`points`	Three or more vertices (`measure_area`), as pairs or a flat run of coordinates

`reference_a`, `reference_b`	The two ends of the known-length reference
`reference_length`	That reference's true length, in `unit_name`
`unit_name`	Unit of the reference length, `mm` by default

Give all three of `reference_a`, `reference_b` and `reference_length` or none of them; a half-stated calibration is rejected rather than guessed at. Without a reference the label reads in pixels, which is honest but rarely what an exhibit needs. `scale_bar` requires one — a bar with no calibration is a ruler with no units.

On the command line the calibration is a *modifier*, shared by every measurement in the chain, because a scale belongs to an image plane rather than to one measurement:

```
python -m src.cli plate.jpg \
   --scale-ref 100,200,340,200 --scale-length 520 --scale-unit mm \
   --measure 40,300,290,300 \
   --measure-area 40,320,290,320,290,400,40,400 \
   --scale-bar 200 -o measured.jpg
```

Area converts by the square of the linear scale, so a calibration that is 5% wrong makes an area about 10% wrong. Read `measure_3d` instead when the thing being measured stands *out* of the calibrated plane — a person's height cannot be had from a scale laid on the ground they are standing on.

Annotation — `arrow`, `text`, `shape`

Presentation rather than measurement: these point a reader at something without claiming anything about it. `shape` draws a `rectangle`, `circle`, `ellipse`, `line` or `polygon`; rectangles, lines, circles and ellipses take exactly two defining points, polygons three or more. Points may be given as pairs or as a flat run of coordinates, so `10,10,60,50` and `[(10, 10), (60, 50)]` mean the same thing.

```
python -m src.cli scene.jpg \
   --arrow start=400,300 end=280,210 label=plate \
   --text text=Exhibit_A position=20,40 \
   --shape shape=rectangle points=260,190,340,240 -o figure.jpg
```

Because these are ordinary chain steps they are recorded in the preset and in the processing report, which is what makes an annotated exhibit reproducible.

`measure_3d` — height out of the ground plane, which the scale above cannot reach. Given the ground plane's horizon, the vanishing point of scene verticals, and one reference object of known height standing on that ground, the height of anything else standing on the same ground follows from a cross-ratio — a quantity projection preserves. The method is Criminisi, Reid and Zisserman, *Single View Metrology*, IJCV 40(2), 2000.

Parameter	Default	Meaning
`base`, `top`	required	Target's ground contact and highest point
`reference_base`, `reference_top`	required	Same two points on the known object

`horizon`	required	A `y` row, `a,b,c`, `x1,y1,x2,y2` on the horizon, or 8 numbers for two receding lines (16 for four)
`reference_height`	`1800.0`	True height of the reference, in `unit_name`
`vertical_point`	`None`	Vertical vanishing point; omit if verticals are parallel
`unit_name`	`'mm'`	Unit label
`show_horizon`	`True`	Draw the horizon the estimate rests on
`show_uncertainty`	`True`	Write the two sensitivities under the height

`measure_height` returns the number without drawing, along with two sensitivities: `uncertainty_per_pixel` for a one-pixel error in the clicked base and top, and `horizon_uncertainty_per_pixel` for a one-pixel shift of the horizon. Read the second first. A base and a top are clicked on features you can see and are rarely more than a pixel or two out; a horizon is *inferred*, and is easily ten pixels out — so the smaller per-pixel figure is usually the larger real error. Both are drawn under the height unless you pass `show_uncertainty=False`.

Neither figure can catch a horizon that is simply in the wrong place. They are local slopes: put the horizon on a ceiling instead of the vanishing point and the reported sensitivity actually *falls*, because a far-off horizon moves the answer little per pixel — while being 170 px wrong moves it enormously. The arithmetic stays self-consistent, so nothing in the five clicked points can flag it. Check the horizon against the scene: it passes through the point where the scene's parallel lines converge.

So don't eyeball it. `horizon_from_lines` takes lines that are *parallel in the scene* — a corridor's floor edge and its ceiling edge, the two kerbs of a road — and returns the horizon through their vanishing point. Lines parallel in the scene meet at a vanishing point in the image, and the vanishing point of any direction parallel to the ground lies on the ground's horizon, so edges you can see give you a horizon you cannot.

Pass one set of lines and the horizon is taken horizontal through that point, which is exact for a camera with no roll. Pass a second set running a different way and it runs through both vanishing points, assuming nothing about levelness. The `horizon` parameter accepts these directly: 8 numbers for two lines, 16 for four.

Pick points on the image collects them — a receding line, then a second one — instead of asking for two points on a horizon you would have to already know. On a real corridor frame, two hand-clicked edges landed within 4 px of a horizon found by Hough transform over the whole image; the same frame with the horizon eyeballed onto the ceiling was 170 px out, worth about 700 mm of height.

`vanishing_point` solves for a vanishing point from two or more scene-parallel lines by least squares, and `horizon_from_vanishing_points` builds the horizon from two of them.

Bases either side of the horizon are refused outright: two objects on one ground plane image on one side of that plane's horizon, so straddling it is impossible rather than merely inaccurate.

The estimate is only as good as its assumptions, and each one fails quietly:

- **Both objects must stand on the same ground plane.** Someone on a kerb, a step or a slope is being measured against a plane they are not on.
- **Correct lens distortion first.** Curved straight lines put the vanishing geometry wrong before any arithmetic happens.

- **The base is the ground contact.** A gap under a heel, or feet hidden behind a car, biases the result directly.
- **Accuracy collapses near the horizon**, where one pixel spans a fast-growing real distance — which is what `uncertainty_per_pixel` is reporting.
- **Omitting `vertical_point` assumes the camera has no tilt.** Against a synthetic camera 2.5 m up with a 1.8 m reference, that assumption over-read by about 16 mm at 5° of pitch and 33 mm at 18°. Most CCTV is tilted, so supply the vertical vanishing point.

Analysis reports (not chain steps)

The measurements above that return numbers rather than an image are registered together in `ANALYSIS_REGISTRY`, beside the filter registry. They never enter a chain: they describe the evidence rather than change it, and running one leaves the pipeline untouched.

Registry name	Function	Module	Reads	CLI
`noise`	`noise_report`	`src.filters.noise_analysis`	pixels	`--noise-stats`
`ela`	`ela_stats`	`src.filters.ela`	pixels	`--ela-stats [QUALITY]`
`clone`	`detect_copy_move`	`src.filters.clone_detection`	pixels	`--clone-stats`
`compression`	`compression_report`	`src.filters.compression_analysis`	pixels + file	`--compression-stats`
`ghost`	`ghost_report`	`src.filters.jpeg_ghost`	pixels	`--ghost-stats`
`metadata`	`metadata_report`	`src.filters.metadata_forensics`	file	`--metadata-stats`

Each entry carries the presentation of its own report — a header line, its rows, and the caveat that closes it — so the CLI prints exactly what the GUI's **Analysis** tab and the dashboard's **Analysis** tab display. The flags in that last column are generated from the same entries, which is why a report added to the registry appears in all three front ends without any of them being edited. `--list-analyses` prints the registered set.

Report	Parameters
`noise`	`block_size=32`
`ela`	`quality=90`, `block_size=16`
`clone`	`block_size=16`, `step=1`, `coefficients=4`, `quantization=4.0`, `min_distance=0.0`, `min_matches=8`, `min_variance=12.0`, `search_window=3`, `max_blocks=300000`
`compression`	`block_size=32` (plus the source path)
`ghost`	`qualities=(50…100 by 5)`, `block_size=16`
`metadata`	none

`compression` and `metadata` read the container, not the chain's output. Quantisation tables and EXIF live in the file on disk, so those two describe the image that was opened however many filters have since been applied — and they have nothing to read at all if the image never came from a file. The GUI says so rather than failing; the dashboard writes the browser's upload to a temporary copy under its own name, so the report still quotes the filename you recognise.

Rows carry a severity: `flag` for a finding worth investigating, `info` for one worth knowing, and nothing for a plain measurement. **None of the three is a conclusion.** Every report ends with a note saying what the measure cannot tell you, and those notes are the short form of the caveats written out under each filter above.

```
from src.filters import report_lines, resolve_analysis, run_analysis

spec = resolve_analysis('ghost')
report = run_analysis(spec, image=pipeline.current, params={'block_size': 8})
print('\n'.join(report_lines(spec, report)))     # what the CLI prints

report['outlier_count']                          # or read the dict directly
```

`run_analysis` supplies whichever of the image and the path a report asks for, and raises `ValueError` rather than guessing when one is missing. `render_report` returns the same rows as `Row(label, value, severity, indent)` objects, for a front end that colours them.

ROI helpers (not chain steps)

`analyze_roi(image, roi)` returns per-channel mean, std, min and max plus pixel count. Exposed on the CLI as `--analyze-roi X,Y,W,H`, which runs against the **processed** image.

`apply_to_roi(image, roi, filter_fn, **kwargs)` applies any filter to a region only, leaving the rest of the frame untouched.

`get_centered_roi(shape, w, h)` and `roi_from_ratio(shape, x, y, w, h)` build regions without hardcoding pixel coordinates.